

《星溯旅人》第四阶段：游戏系统开发 - 详细分步操作指南

部分内容由豆包生成



本阶段目标：实现完整的游戏玩法系统，包括三大核心特色系统（时间褶皱、星核共鸣、生态共生）、战斗系统、技能系统、AI系统等

预计耗时：4-6周

前置条件：第三阶段渲染系统开发已完成

一、第四阶段概述

1.1 阶段目标

第四阶段是游戏开发的**核心玩法阶段**，将实现《星溯旅人》的全部游戏系统。这一阶段完成后，游戏将具备完整的可玩体验。

1.2 主要工作内容

序号	系统	主要工作
1	玩家系统	角色移动、属性、状态机、成长系统
2	敌人系统	敌人基类、类型、行为模式、掉落
3	战斗系统	伤害计算、受击反馈、战斗状态、连击
4	时间褶皱系统	时间减速/加速、时间回溯、时间悖论
5	星核共鸣系统	7种共鸣类型、能力解锁、反噬机制
6	生态共生系统	生物图鉴、友好度、伙伴系统、协同效果
7	技能系统	技能树、冷却、技能效果、升级
8	AI系统	感知系统、行为树、状态机、路径规划
9	背包道具系统	物品管理、装备系统、商店、合成

10	任务系统	任务链、进度追踪、奖励、对话
11	集成测试	玩法测试、平衡性测试、压力测试

1.3 三大核心特色系统



时间褶皱

- 时间流速控制
- 时间回溯
- 时间谜题
- 悖论系统



星核共鸣

- 7种共鸣类型
- 环境操控能力
- 反噬风险
- 星核人格



生态共生

- 共生关系
- 进化路线
- 生态平衡
- 动态事件

二、步骤1：完善玩家角色系统



目标：完善玩家角色系统，实现完整的角色控制、属性、状态机和成长系统

修改文件：player.h / player.cpp

重要性：★★★★★（所有玩法的基础）

1.1 玩家系统功能

- **基础属性：**生命值、能量、等级、经验、攻击力、防御力
- **移动系统：**8方向移动、冲刺、跳跃（2.5D）
- **状态机：**待机、移动、攻击、受击、死亡、跳跃等状态
- **成长系统：**升级、属性成长、技能点
- **buff/debuff：**增益和减益效果

1.2 完善Player头文件

操作步骤

1. 在Qt Creator中打开 `src/game/player.h`
2. 添加玩家状态枚举
3. 添加更多属性（攻击、防御、暴击等）

4. 添加状态机相关方法

5. 添加buff系统

6. 保存文件

player.h 核心新增内容

player.h - 新增内容

```
1  // 玩家状态枚举
2  enum class PlayerState {
3      Idle,           // 待机
4      Walking,        // 行走
5      Running,        // 奔跑
6      Jumping,        // 跳跃
7      Falling,        // 下落
8      Attacking,      // 攻击中
9      Hurt,           // 受击
10     Dashing,         // 冲刺
11     Crouching,       // 蹲下
12     Dead,           // 死亡
13     Count
14 };
15
16 // 属性类型
17 enum class AttributeType {
18     Health,          // 生命值
19     Energy,          // 能量
20     Attack,          // 攻击力
21     Defense,         // 防御力
22     Speed,           // 移动速度
23     CriticalRate,     // 暴击率
24     CriticalDamage,  // 暴击伤害
25     Count
26 };
27
28 // Buff结构体
29 struct Buff {
30     QString id;
31     QString name;
32     AttributeType attribute;
33     float value;      // 加成值（百分比或绝对值）
34     bool isPercentage; // 是否为百分比加成
35     float duration;    // 持续时间（秒）
36     float remainingTime; // 剩余时间
37     bool isActive;
38 };
```

```

39
40 // 新增方法:
41 // 状态管理
42 PlayerState getState() const;
43 void setState(PlayerState state);
44 bool isState(PlayerState state) const;
45 bool canAct() const; // 是否可以行动 (非攻击、非受击等)
46
47 // 战斗属性
48 int getAttack() const;
49 int getDefense() const;
50 float getCriticalRate() const;
51 float getCriticalDamage() const;
52
53 // 移动控制
54 void setMovementDirection(const QPointF& direction);
55 QPointF getMovementDirection() const;
56 void dash();
57 bool isDashing() const;
58
59 // 跳跃 (2.5D)
60 void jump();
61 bool isGrounded() const;
62 void setGrounded(bool grounded);
63 float getJumpHeight() const;
64
65 // Buff系统
66 bool addBuff(const Buff& buff);
67 bool removeBuff(const QString& buffId);
68 bool hasBuff(const QString& buffId) const;
69 QList<Buff> getActiveBuffs() const;
70 void updateBuffs(float deltaTime);
71
72 // 升级系统
73 int getSkillPoints() const;
74 void addSkillPoints(int points);
75 bool spendSkillPoint(AttributeType attribute);
76
77 // 经验和升级
78 int getExpToNextLevel() const;
79 void checkLevelUp();

```

1.3 完善Player实现

重点实现：状态机

玩家状态机实现

```
void Player::update(float deltaTime)
2  {
3      if (!m_isInitialized || m_state == PlayerState::Dead) {
4          return;
5      }
6
7      // 更新Buff
8      updateBuffs(deltaTime);
9
10     // 根据当前状态执行不同逻辑
11     switch (m_state) {
12         case PlayerState::Idle:
13             updateIdle(deltaTime);
14             break;
15         case PlayerState::Walking:
16         case PlayerState::Running:
17             updateMovement(deltaTime);
18             break;
19         case PlayerState::Jumping:
20         case PlayerState::Falling:
21             updateJump(deltaTime);
22             break;
23         case PlayerState::Attacking:
24             updateAttack(deltaTime);
25             break;
26         case PlayerState::Hurt:
27             updateHurt(deltaTime);
28             break;
29         case PlayerState::Dashing:
30             updateDash(deltaTime);
31             break;
32         default:
33             break;
34     }
35
36     // 能量恢复
37     if (m_state != PlayerState::Dead) {
38         m_energy = qMin(m_maxEnergy, m_energy + m_energyRegenRate * deltaTime);
39     }
40 }
41
42 void Player::setState(PlayerState state)
43 {
44     if (m_state == state) {
45         return;
46     }
47 }
```

```

48     // 退出旧状态
49     onStateExit(m_state);
50
51     PlayerState oldState = m_state;
52     m_state = state;
53
54     // 进入新状态
55     onStateEnter(state);
56
57     emit stateChanged(static_cast<int>(oldState), static_cast<int>(state));
58 }
59
60 void Player::onStateEnter(PlayerState state)
61 {
62     switch (state) {
63         case PlayerState::Idle:
64             m_movementDirection = QPointF(0, 0);
65             break;
66         case PlayerState::Hurt:
67             m_hurtTimer = m_hurtDuration;
68             m_invincible = true;
69             break;
70         case PlayerState::Dashing:
71             m_dashTimer = m_dashDuration;
72             m_invincible = true;
73             break;
74         default:
75             break;
76     }
77 }
78
79 void Player::onStateExit(PlayerState state)
80 {
81     switch (state) {
82         case PlayerState::Hurt:
83             m_invincible = false;
84             break;
85         case PlayerState::Dashing:
86             m_invincible = false;
87             break;
88         default:
89             break;
90     }
91 }

```

重点实现：移动系统

移动系统实现

```
1 void Player::updateMovement(float deltaTime)
2 {
3     if (m_movementDirection.isNull()) {
4         setState(PlayerState::Idle);
5         return;
6     }
7
8     // 归一化方向
9     float length = qSqrt(m_movementDirection.x() * m_movementDirection.x() +
10                          m_movementDirection.y() * m_movementDirection.y());
11     if (length > 0.0f) {
12         m_movementDirection /= length;
13     }
14
15     // 计算实际速度（考虑Buff加成）
16     float actualSpeed = getActualSpeed();
17
18     // 移动
19     QPointF movement = m_movementDirection * actualSpeed * deltaTime;
20     m_position += movement;
21
22     // 更新朝向
23     if (qAbs(m_movementDirection.x()) > 0.1f) {
24         m_facingRight = m_movementDirection.x() > 0;
25     }
26
27     emit positionChanged(m_position);
28 }
29
30 float Player::getActualSpeed() const
31 {
32     float speed = m_speed;
33
34     // 应用Buff加成
35     for (const Buff& buff : m_activeBuffs) {
36         if (buff.attribute == AttributeType::Speed && buff.isActive) {
37             if (buff.isPercentage) {
38                 speed *= (1.0f + buff.value);
39             } else {
40                 speed += buff.value;
41             }
42         }
43     }
44
45     return speed;
```

重点实现：升级系统

升级系统实现

```
1  void Player::gainExperience(int amount)
2  {
3      if (amount <= 0) {
4          return;
5      }
6
7      m_experience += amount;
8      emit experienceChanged(m_experience);
9
10     // 检查是否升级
11     checkLevelUp();
12 }
13
14 void Player::checkLevelUp()
15 {
16     while (m_experience >= getExpToNextLevel()) {
17         m_experience -= getExpToNextLevel();
18         m_level++;
19
20         // 升级属性提升
21         m_maxHealth += 10;
22         m_health = m_maxHealth;
23         m_maxEnergy += 5;
24         m_energy = m_maxEnergy;
25         m_attack += 2;
26         m_defense += 1;
27
28         // 获得技能点
29         m_skillPoints += 1;
30
31         qDebug() << "[Player] 升级! 当前等级:" << m_level;
32         emit levelUp(m_level);
33     }
34 }
35
36 int Player::getExpToNextLevel() const
37 {
38     // 经验需求随等级增长
39     return static_cast<int>(100 * qPow(1.5f, m_level - 1));
```


验收标准

- ☐ 编译无错误
- ☐ 玩家移动控制正常
- ☐ 状态机切换正确
- ☐ 属性系统正常
- ☐ 升级系统正常
- ☐ Buff系统正常
- ☐ 受击和无敌帧正常

三、步骤2：实现敌人系统



目标：实现敌人系统，包括敌人基类、多种敌人类型、行为模式和掉落系统

修改文件：enemy.h / enemy.cpp

新增文件：enemyTypes.h（敌人类型定义）

2.1 敌人系统功能

- **敌人基类：**通用属性和行为
- **敌人类型：**普通怪、精英怪、Boss等
- **AI行为：**巡逻、追击、攻击、逃跑
- **掉落系统：**经验、金币、道具掉落
- **仇恨系统：**仇恨值、仇恨目标

2.2 敌人类型设计

类型	特点	行为模式	示例
普通敌人	低血量、低攻击	巡逻 + 追击	史莱姆、小机器人
远程敌人	远程攻击、脆皮	保持距离射击	炮台、弓箭手
精英敌人	高血量、高攻击	多种攻击模式	强化机器人

Boss	超高血量、多阶段	复杂AI、技能	区域Boss
中立生物	不主动攻击	逃跑、友好	生态生物

2.3 完善Enemy头文件

enemy.h - 核心内容

```
1  // 敌人AI状态
2  enum class EnemyAIState {
3      Idle,           // 待机
4      Patrol,        // 巡逻
5      Chase,          // 追击
6      Attack,         // 攻击
7      Hurt,           // 受击
8      Flee,            // 逃跑
9      Dead,           // 死亡
10     Special         // 特殊行为
11 };
12
13 // 敌人类型
14 enum class EnemyType {
15     Normal,          // 普通敌人
16     Ranged,           // 远程敌人
17     Elite,            // 精英敌人
18     Boss,             // Boss
19     Neutral,          // 中立生物
20     Count
21 };
22
23 // 掉落物品
24 struct DropItem {
25     QString itemId;
26     int minAmount;
27     int maxAmount;
28     float dropRate; // 掉落概率 0-1
29 };
30
31 class Enemy : public QObject
32 {
33     Q_OBJECT
34 public:
35     explicit Enemy(QObject* parent = nullptr);
36     ~Enemy() override;
37
38     bool initialize(const QString& enemyId, EnemyType type);
```

```
39     void update(float deltaTime);
40
41     // 属性
42     QString getId() const;
43     QString getName() const;
44     EnemyType getType() const;
45     int getHealth() const;
46     int getMaxHealth() const;
47     int getAttack() const;
48     int getDefense() const;
49     float getSpeed() const;
50     QPointF getPosition() const;
51     void setPosition(const QPointF& pos);
52
53     // AI状态
54     EnemyAIState getAIState() const;
55     void setAIState(EnemyAIState state);
56
57     // 仇恨系统
58     void addThreat(const QString& targetId, float amount);
59     QString getTopThreatTarget() const;
60     void clearThreat();
61     float getDetectionRange() const;
62     float getAttackRange() const;
63
64     // 战斗
65     void takeDamage(int damage, const QString& attackerId);
66     bool isAlive() const;
67     bool canAttack() const;
68     float getAttackCooldown() const;
69
70     // 掉落
71     QList<DropItem> getDrops() const;
72     int getExpReward() const;
73     int getGoldReward() const;
74
75     signals:
76     void healthChanged(int current, int max);
77     void enemyDied(const QString& enemyId);
78     void stateChanged(EnemyAIState newState);
79     void attackRequested(const QString& enemyId, const QString& targetId);
80
81     private:
82     // AI更新
83     void updateAI(float deltaTime);
84     void updateIdle(float deltaTime);
85     void updatePatrol(float deltaTime);
```

```
86     void updateChase(float deltaTime);
87     void updateAttack(float deltaTime);
88     void updateHurt(float deltaTime);
89
90     // 感知
91     bool detectPlayer();
92     float getDistanceToPlayer() const;
93
94     // 成员变量
95     QString m_id;
96     QString m_name;
97     EnemyType m_type;
98     EnemyAIState m_aiState;
99
100    // 属性
101    int m_health;
102    int m_maxHealth;
103    int m_attack;
104    int m_defense;
105    float m_speed;
106
107    // 位置
108    QPointF m_position;
109
110    // AI参数
111    float m_detectionRange;    // 检测范围
112    float m_attackRange;      // 攻击范围
113    float m_patrolRadius;     // 巡逻半径
114    QPointF m_homePosition;   // home位置
115
116    // 仇恨
117    QMap<QString, float> m_threatTable;
118
119    // 攻击冷却
120    float m_attackCooldown;
121    float m_currentCooldown;
122
123    // 受击
124    float m_hurtDuration;
125    float m_hurtTimer;
126
127    // 掉落
128    QList<DropItem> m_drops;
129    int m_expReward;
130    int m_goldReward;
131
132    bool m_isInitialized;
```

2.4 敌人管理器

创建敌人管理器来管理场景中的所有敌人：

```
EnemyManager

1  class EnemyManager : public QObject
2  {
3      Q_OBJECT
4  public:
5      explicit EnemyManager(QObject* parent = nullptr);
6      ~EnemyManager() override;
7
8      bool initialize();
9      void update(float deltaTime);
10
11     // 敌人管理
12     Enemy* spawnEnemy(const QString& enemyId, EnemyType type, const QPointF&
position);
13     void removeEnemy(const QString& enemyId);
14     Enemy* getEnemy(const QString& enemyId) const;
15     QList<Enemy*> getAllEnemies() const;
16     int getEnemyCount() const;
17
18     // 区域管理
19     void setActiveArea(const QRectF& area);
20     QList<Enemy*> getEnemiesInArea(const QRectF& area) const;
21
22     signals:
23         void enemySpawned(const QString& enemyId);
24         void enemyDied(const QString& enemyId);
25
26     private:
27         QMap<QString, Enemy*> m_enemies;
28         QRectF m_activeArea;
29         bool m_isInitialized;
30 };
```

验收标准

- ☐ 编译无错误
- ☐ 敌人生成和销毁正常

- ☐ 敌人AI行为正常
- ☐ 仇恨系统正常
- ☐ 敌人受击和死亡正常
- ☐ 掉落系统正常
- ☐ 多种敌人类型可配置

四、步骤3：实现战斗系统



目标：实现完整的战斗系统，包括伤害计算、受击反馈、战斗状态、连击系统等

修改文件：combatSystem.h / combatSystem.cpp

重要性：★★★★★（核心玩法）

3.1 战斗系统功能

- **伤害计算：**基础伤害、防御减免、暴击、属性克制
- **受击反馈：**击退、无敌帧、硬直
- **战斗状态：**进入/退出战斗、战斗音乐
- **连击系统：**连击计数、连击奖励
- **伤害数字：**飘字效果

3.2 伤害计算公式

伤害计算公式

```
1 // 最终伤害 = (基础伤害 × 技能倍率 × 属性克制 - 防御值) × 暴击倍率 × 其他加成
2
3 // 详细计算步骤：
4 float CombatSystem::calculateDamage(const DamageInfo& info) const
5 {
6     float damage = info.baseDamage;
7
8     // 1. 技能倍率
9     damage *= info.skillMultiplier;
10
11     // 2. 属性克制
12     damage *= getElementMultiplier(info.attackElement, info.defenseElement);
13
14     // 3. 防御减免（防御值减伤公式）
```

```

15     float defenseReduction = info.defense / (info.defense + 100.0f);
16     damage *= (1.0f - defenseReduction);
17
18     // 4. 暴击判定
19     bool isCritical = false;
20     if (info.criticalRate > 0.0f) {
21         float roll = static_cast<float>(qrand()) / RAND_MAX;
22         isCritical = roll < info.criticalRate;
23     }
24
25     if (isCritical) {
26         damage *= info.criticalMultiplier;
27     }
28
29     // 5. 随机浮动 (±10%)
30     float variance = 0.9f + static_cast<float>(qrand()) / RAND_MAX * 0.2f;
31     damage *= variance;
32
33     // 6. 确保最小伤害为1
34     damage = qMax(1.0f, damage);
35
36     return damage;
37 }
38
39 // 属性克制关系
40 float CombatSystem::getElementMultiplier(ElementType attack, ElementType
defense) const
41 {
42     // 火克风、风克地、地克水、水克火
43     // 光暗互克
44     static const float multiplierTable[7][7] = {
45         // 火    水    风    地    光    暗    无
46         { 1.0f, 0.5f, 1.5f, 1.0f, 1.0f, 1.0f, 1.0f }, // 火
47         { 1.5f, 1.0f, 1.0f, 0.5f, 1.0f, 1.0f, 1.0f }, // 水
48         { 0.5f, 1.0f, 1.0f, 1.5f, 1.0f, 1.0f, 1.0f }, // 风
49         { 1.0f, 1.5f, 0.5f, 1.0f, 1.0f, 1.0f, 1.0f }, // 地
50         { 1.0f, 1.0f, 1.0f, 1.0f, 1.0f, 1.5f, 1.0f }, // 光
51         { 1.0f, 1.0f, 1.0f, 1.0f, 1.5f, 1.0f, 1.0f }, // 暗
52         { 1.0f, 1.0f, 1.0f, 1.0f, 1.0f, 1.0f, 1.0f }, // 无
53     };
54
55     return multiplierTable[static_cast<int>(attack)][static_cast<int>
(defense)];
56 }

```

3.3 战斗系统实现要点

受击反馈

受击反馈实现

```
1 void CombatSystem::applyDamage(DamageInfo& info)
2 {
3     // 1. 计算伤害
4     int finalDamage = static_cast<int>(calculateDamage(info));
5
6     // 2. 应用伤害
7     info.target->takeDamage(finalDamage);
8
9     // 3. 受击击退
10    if (info.knockback > 0.0f) {
11        QPointF knockbackDir = info.target->getPosition() - info.attacker-
12        >getPosition();
13        // 归一化
14        float length = qSqrt(knockbackDir.x() * knockbackDir.x() +
15                             knockbackDir.y() * knockbackDir.y());
16        if (length > 0.0f) {
17            knockbackDir /= length;
18        }
19        info.target->applyKnockback(knockbackDir * info.knockback);
20    }
21
22    // 4. 屏幕震动（根据伤害大小）
23    float shakeIntensity = qMin(10.0f, finalDamage / 10.0f);
24    m_camera->shake(shakeIntensity, 0.2f);
25
26    // 5. 伤害数字飘字
27    showDamageNumber(info.target->getPosition(), finalDamage, info.isCritical);
28
29    // 6. 更新连击
30    if (info.isPlayerAttack) {
31        updateCombo();
32    }
33
34    // 7. 战斗统计
35    if (info.isPlayerAttack) {
36        m_totalDamageDealt += finalDamage;
37    } else {
38        m_totalDamageTaken += finalDamage;
39    }
40
41    emit damageDealt(finalDamage, info.isCritical);
42 }
```


连击系统

连击系统

```
1  void CombatSystem::updateCombo()
2  {
3      m_comboCount++;
4      m_comboTimer = m_comboTimeout;
5
6      // 连击奖励
7      if (m_comboCount >= 10 && !m_combo10Bonus) {
8          // 10连击奖励
9          m_combo10Bonus = true;
10         emit comboMilestone(10);
11     }
12     // ... 更多连击里程碑
13
14     emit comboChanged(m_comboCount);
15 }
16
17 void CombatSystem::update(float deltaTime)
18 {
19     // 连击计时
20     if (m_comboCount > 0) {
21         m_comboTimer -= deltaTime;
22         if (m_comboTimer <= 0.0f) {
23             resetCombo();
24         }
25     }
26
27     // 战斗状态检查
28     if (m_isInCombat) {
29         m_combatTimer -= deltaTime;
30         if (m_combatTimer <= 0.0f) {
31             exitCombat();
32         }
33     }
34 }
35
36 float CombatSystem::getComboDamageBonus() const
37 {
38     // 连击伤害加成：每10连击+5%伤害，最多+50%
39     float bonus = (m_comboCount / 10) * 0.05f;
40     return qMin(0.5f, bonus);
41 }
```

验收标准

- ☐ 编译无错误
- ☐ 伤害计算正确
- ☐ 暴击系统正常
- ☐ 属性克制正确
- ☐ 受击反馈正常
- ☐ 连击系统正常
- ☐ 战斗状态切换正常

五、步骤4：实现时间褶皱系统



目标：实现时间褶皱系统——游戏三大核心特色系统之一

修改文件：timeSystem.h / timeSystem.cpp

重要性：★★★★★（核心特色系统）

4.1 时间褶皱系统功能

- **时间流速控制：**减速、加速、暂停
- **时间回溯：**倒回时间，撤销操作
- **时间悖论：**过度使用时间能力的反噬
- **时间谜题：**利用时间能力解谜
- **时间感知：**玩家不受时间影响

4.2 时间流速控制

时间流速控制

```
1 // 时间流速档位
2 enum class TimeScaleLevel {
3     Paused = 0,           // 暂停
4     VerySlow = 0.25f,    // 极慢 (25%)
5     Slow = 0.5f,         // 慢速 (50%)
6     Normal = 1.0f,       // 正常
7     Fast = 2.0f,         // 快速 (200%)
8     VeryFast = 4.0f      // 极快 (400%)
9 };
```

```
10
11 void TimeSystem::setTimeScale(float scale)
12 {
13     if (qAbs(m_timeScale - scale) < 0.001f) {
14         return;
15     }
16
17     m_timeScale = scale;
18
19     // 通知游戏引擎更新时间缩放
20     // GameEngine::instance()->setTimeScale(scale);
21
22     qDebug() << "[TimeSystem] 时间流速设置为:" << scale;
23     emit timeScaleChanged(scale);
24 }
25
26 void TimeSystem::slowTime(float amount, float duration)
27 {
28     // 时间减速效果
29     m_timeScale = amount;
30     m_slowTimer = duration;
31     m_isSlowActive = true;
32
33     // 玩家不受影响（玩家时间正常流逝）
34     // 只有世界时间减速
35
36     emit timeSlowed();
37 }
38
39 void TimeSystem::update(float deltaTime)
40 {
41     if (!m_isInitialized) {
42         return;
43     }
44
45     // 更新减速计时
46     if (m_isSlowActive) {
47         m_slowTimer -= deltaTime;
48         if (m_slowTimer <= 0.0f) {
49             m_isSlowActive = false;
50             m_timeScale = 1.0f;
51             emit timeResumed();
52         }
53     }
54
55     // 更新回溯
56     if (m_isRewinding) {
```

```

57         updateRewind(deltaTime);
58     } else {
59         // 记录历史状态（用于回溯）
60         recordHistory(deltaTime);
61     }
62
63     // 更新悖论值
64     updateParadox(deltaTime);
65 }

```

4.3 时间回溯系统

时间回溯实现

```

1  // 历史状态记录
2  struct TimeHistoryFrame {
3      float timestamp;
4      QPointF playerPosition;
5      int playerHealth;
6      int playerEnergy;
7      // ... 更多需要回溯的状态
8  };
9
10 bool TimeSystem::canRewind() const
11 {
12     // 有足够的历史记录，且有足够能量
13     return m_historyBuffer.size() > 10 &&
14           m_rewindEnergy > m_rewindCostPerSecond;
15 }
16
17 void TimeSystem::startRewind()
18 {
19     if (!canRewind()) {
20         return;
21     }
22
23     m_isRewinding = true;
24     m_rewindSpeed = 2.0f; // 2倍速回溯
25
26     // 时间暂停（除了玩家）
27     setTimeScale(0.0f);
28
29     qDebug() << "[TimeSystem] 开始时间回溯";
30     emit rewindStarted();
31 }
32

```

```
33 void TimeSystem::stopRewind()
34 {
35     if (!m_isRewinding) {
36         return;
37     }
38
39     m_isRewinding = false;
40
41     // 恢复正常时间
42     setTimeScale(1.0f);
43
44     // 清空已回溯的历史
45     // 保留当前时间点之后的历史（不，应该是丢弃已回溯的部分）
46
47     // 增加悖论值
48     addParadox(m_rewindParadoxCost);
49
50     qDebug() << "[TimeSystem] 停止时间回溯";
51     emit rewindStopped();
52 }
53
54 void TimeSystem::updateRewind(float deltaTime)
55 {
56     // 消耗回溯能量
57     m_rewindEnergy -= m_rewindCostPerSecond * deltaTime * m_rewindSpeed;
58
59     if (m_rewindEnergy <= 0.0f || m_historyBuffer.isEmpty()) {
60         stopRewind();
61         return;
62     }
63
64     // 从历史缓冲区中取出状态并恢复
65     float rewindAmount = deltaTime * m_rewindSpeed;
66
67     while (rewindAmount > 0 && !m_historyBuffer.isEmpty()) {
68         TimeHistoryFrame frame = m_historyBuffer.takeLast();
69         // 恢复状态...
70         rewindAmount -= 0.016f; // 假设每帧16ms
71     }
72 }
73
74 void TimeSystem::recordHistory(float deltaTime)
75 {
76     m_recordTimer += deltaTime;
77
78     // 每16ms记录一帧（约60fps）
79     if (m_recordTimer >= 0.016f) {
```

```

80         m_recordTimer -= 0.016f;
81
82         TimeHistoryFrame frame;
83         frame.timestamp = m_totalTime;
84         // frame.playerPosition = m_player->getPosition();
85         // ... 记录其他状态 ...
86
87         m_historyBuffer.append(frame);
88
89         // 限制历史记录长度（最多回溯5秒）
90         while (m_historyBuffer.size() > 300) { // 5秒 × 60帧
91             m_historyBuffer.removeFirst();
92         }
93     }
94 }

```

4.4 时间悖论系统

过度使用时间能力会产生时间悖论，产生负面效果：

时间悖论系统

```

1  // 悖论等级
2  enum class ParadoxLevel {
3      None,          // 无悖论
4      Mild,          // 轻微（视觉干扰）
5      Moderate,      // 中等（属性下降）
6      Severe,        // 严重（操作异常）
7      Critical       // 危险（持续掉血）
8  };
9
10 void TimeSystem::addParadox(float amount)
11 {
12     m_paradoxValue = qMin(100.0f, m_paradoxValue + amount);
13     updateParadoxLevel();
14     emit paradoxChanged(m_paradoxValue);
15 }
16
17 void TimeSystem::updateParadox(float deltaTime)
18 {
19     // 悖论值随时间缓慢恢复
20     if (m_paradoxValue > 0.0f && !m_isRewinding) {
21         m_paradoxValue = qMax(0.0f, m_paradoxValue - m_paradoxRecoveryRate *
22         deltaTime);
23         updateParadoxLevel();
24     }
25 }

```

```

24 }
25
26 void TimeSystem::updateParadoxLevel()
27 {
28     ParadoxLevel oldLevel = m_paradoxLevel;
29
30     if (m_paradoxValue >= 80.0f) {
31         m_paradoxLevel = ParadoxLevel::Critical;
32     } else if (m_paradoxValue >= 60.0f) {
33         m_paradoxLevel = ParadoxLevel::Severe;
34     } else if (m_paradoxValue >= 30.0f) {
35         m_paradoxLevel = ParadoxLevel::Moderate;
36     } else if (m_paradoxValue >= 10.0f) {
37         m_paradoxLevel = ParadoxLevel::Mild;
38     } else {
39         m_paradoxLevel = ParadoxLevel::None;
40     }
41
42     if (m_paradoxLevel != oldLevel) {
43         applyParadoxEffects();
44         emit paradoxLevelChanged(static_cast<int>(m_paradoxLevel));
45     }
46 }
47
48 void TimeSystem::applyParadoxEffects()
49 {
50     // 根据悖论等级应用不同效果
51     switch (m_paradoxLevel) {
52         case ParadoxLevel::Mild:
53             // 轻微：屏幕闪烁、音效异常
54             break;
55         case ParadoxLevel::Moderate:
56             // 中等：属性下降10%、输入延迟
57             break;
58         case ParadoxLevel::Severe:
59             // 严重：操作反转、时间随机波动
60             break;
61         case ParadoxLevel::Critical:
62             // 危险：持续掉血、视野混乱
63             break;
64         default:
65             break;
66     }
67 }

```

验收标准

- ☐ 编译无错误
- ☐ 时间减速/加速功能正常
- ☐ 时间暂停功能正常
- ☐ 时间回溯功能正常
- ☐ 悖论系统正常
- ☐ 玩家不受时间影响
- ☐ 能量消耗正常

六、步骤5：实现星核共鸣系统

- ★

目标：实现星核共鸣系统——游戏三大核心特色系统之二
- 修改文件：resonanceSystem.h / resonanceSystem.cpp
- 重要性：★★★★★（核心特色系统）

5.1 星核共鸣系统功能

- 7种共鸣类型：光、暗、火、水、风、地、无
- 星核收集：探索获取星核碎片
- 共鸣等级：提升共鸣解锁新能力
- 环境操控：利用共鸣改变环境
- 反噬风险：过度使用共鸣的副作用
- 星核人格：高等级后星核产生意识

5.2 7种共鸣类型

共鸣类型	代表色	核心能力	反噬效果
光之共鸣	金色	照明、净化、治疗	视觉过曝、失明
暗之共鸣	紫色	隐身、暗影攻击	精神侵蚀、恐惧
火之共鸣	红色	火焰攻击、融化障碍	过热、灼烧伤害
水之共鸣	蓝色	冰冻、水流操控	体温过低、减速

风之共鸣	青色	加速、飞行、风刃	失重、方向感丧失
地之共鸣	棕色	防御、地震、操控岩石	沉重、移动困难
无之共鸣	白色	平衡、中和其他属性	虚无感、存在淡化

5.3 星核共鸣系统实现

星核共鸣系统实现

```
1 // 共鸣类型枚举
2 enum class ResonanceType {
3     Light,      // 光
4     Dark,       // 暗
5     Fire,       // 火
6     Water,      // 水
7     Wind,       // 风
8     Earth,      // 地
9     None,       // 无
10    Count
11 };
12
13 // 星核结构体
14 struct StarCore {
15     QString id;
16     QString name;
17     ResonanceType type;
18     int level;
19     int maxLevel;
20     float resonanceValue;    // 当前共鸣值
21     float resonanceCapacity; // 共鸣容量
22     bool isActive;
23     bool isUnlocked;
24
25     // 能力列表
26     QStringList abilities;
27 };
28
29 // 反噬效果
30 struct ResonanceBacklash {
31     ResonanceType type;
32     float intensity;    // 0-1
33     float duration;
34     QString effectId;
35 };
36
37 class ResonanceSystem : public QObject
```

```
38 {
39     Q_OBJECT
40 public:
41     explicit ResonanceSystem(QObject* parent = nullptr);
42     ~ResonanceSystem() override;
43
44     bool initialize();
45     void update(float deltaTime);
46
47     // 星核管理
48     bool unlockStarCore(ResonanceType type, const QString& name);
49     bool hasStarCore(ResonanceType type) const;
50     StarCore getStarCore(ResonanceType type) const;
51     QList<StarCore> getAllStarCores() const;
52
53     // 共鸣值
54     float getResonance(ResonanceType type) const;
55     float getTotalResonance() const;
56     void addResonance(ResonanceType type, float amount);
57     void consumeResonance(ResonanceType type, float amount);
58     bool hasEnoughResonance(ResonanceType type, float amount) const;
59
60     // 激活/切换
61     bool activateStarCore(ResonanceType type);
62     bool deactivateStarCore(ResonanceType type);
63     ResonanceType getActiveResonance() const;
64     bool isResonanceActive(ResonanceType type) const;
65
66     // 等级系统
67     int getResonanceLevel(ResonanceType type) const;
68     bool levelUpResonance(ResonanceType type);
69     float getLevelProgress(ResonanceType type) const;
70
71     // 能力系统
72     QStringList getUnlockedAbilities(ResonanceType type) const;
73     bool isAbilityUnlocked(const QString& abilityId) const;
74     bool useAbility(const QString& abilityId);
75
76     // 反噬系统
77     float getBacklashIntensity(ResonanceType type) const;
78     void addBacklash(ResonanceType type, float amount);
79     bool isBacklashActive(ResonanceType type) const;
80
81 signals:
82     void starCoreUnlocked(ResonanceType type);
83     void resonanceChanged(ResonanceType type, float value);
84     void resonanceLevelUp(ResonanceType type, int newLevel);
```

```

85     void abilityUnlocked(const QString& abilityId);
86     void backlashStarted(ResonanceType type);
87     void backlashEnded(ResonanceType type);
88
89 private:
90     // 更新反噬
91     void updateBacklash(float deltaTime);
92
93     // 检查能力解锁
94     void checkAbilityUnlocks(ResonanceType type);
95
96     // 成员变量
97     QMap<ResonanceType, StarCore> m_starCores;
98     ResonanceType m_activeResonance;
99
100    // 反噬
101    QMap<ResonanceType, ResonanceBacklash> m_backlashEffects;
102
103    bool m_isInitialized;
104 };

```

5.4 环境交互

星核共鸣的核心玩法是与环境互动：

环境交互示例

```

1  // 火之共鸣：点燃火把、融化冰块
2  bool ResonanceSystem::interactWithEnvironment(ResonanceType type, const
  QString& targetId)
3  {
4      if (!isResonanceActive(type)) {
5          return false;
6      }
7
8      // 检查共鸣值是否足够
9      float cost = getInteractionCost(type, targetId);
10     if (!hasEnoughResonance(type, cost)) {
11         return false;
12     }
13
14     // 消耗共鸣值
15     consumeResonance(type, cost);
16
17     // 执行交互
18     switch (type) {

```

```
19         case ResonanceType::Fire:
20             // 点燃、融化
21             break;
22         case ResonanceType::Water:
23             // 冰冻、熄灭火焰
24             break;
25         case ResonanceType::Wind:
26             // 吹动物体、加速
27             break;
28         case ResonanceType::Earth:
29             // 升起平台、创造障碍
30             break;
31         // ... 其他类型
32     }
33
34     // 增加反噬
35     addBacklash(type, cost * 0.1f);
36
37     return true;
38 }
```

验收标准

- ☐ 编译无错误
- ☐ 7种共鸣类型都已定义
- ☐ 星核解锁和升级正常
- ☐ 共鸣值消耗和恢复正常
- ☐ 能力解锁正常
- ☐ 反噬系统正常
- ☐ 环境交互功能正常

七、步骤6：实现生态共生系统



目标：实现生态共生系统——游戏三大核心特色系统之三

修改文件：ecosystemSystem.h / ecosystemSystem.cpp

重要性：★★★★★（核心特色系统）

6.1 生态共生系统功能

- **生物图鉴**：发现和记录外星生物
- **友好度系统**：与生物建立关系
- **伙伴系统**：携带生物伙伴共同冒险
- **共生能力**：伙伴提供的特殊能力
- **进化路线**：生物的成长和进化
- **生态平衡**：区域生态系统的动态平衡

6.2 生物类型

类型	特点	共生能力	友好方式
植物型	静止、光合作用	治疗、净化	浇水、晒太阳
动物型	移动、有情绪	战斗辅助、探索	喂食、抚摸、玩耍
矿物型	坚硬、能量核心	防御、共鸣增幅	打磨、充能
能量型	虚幻、元素属性	元素攻击、特殊效果	提供对应元素
机械型	可编程、可改造	工具、维修、黑客	维修、升级

6.3 生态共生系统实现

```
生态共生系统实现

1  // 生物类型
2  enum class CreatureType {
3      Plant,        // 植物型
4      Animal,       // 动物型
5      Mineral,      // 矿物型
6      Energy,       // 能量型
7      Mechanic,     // 机械型
8      Count
9  };
10
11 // 友好度等级
12 enum class FriendshipLevel {
13     Stranger,    // 陌生
14     Acquainted, // 认识
15     Friendly,   // 友好
16     Trusted,    // 信任
17     Partner,    // 伙伴
18 }
```

```
18     Soulbound    // 灵魂绑定
19 };
20
21 // 生物结构体
22 struct Creature {
23     QString id;
24     QString name;
25     QString description;
26     CreatureType type;
27
28     // 图鉴
29     bool isDiscovered;
30     bool isDetailed; // 是否有详细信息
31
32     // 友好度
33     int friendshipPoints;
34     FriendshipLevel friendshipLevel;
35
36     // 伙伴系统
37     bool canBeCompanion;
38     bool isCompanion;
39     int companionSlot;
40
41     // 能力
42     QStringList abilities;
43     QString activeAbility;
44
45     // 进化
46     int evolutionStage;
47     QStringList evolutionPath;
48     bool canEvolve;
49
50     // 状态
51     float health;
52     float energy;
53     float mood;
54 };
55
56 // 生态区域
57 struct EcosystemZone {
58     QString id;
59     QString name;
60     QRectF area;
61     float balance; // 生态平衡值 0-1
62     QStringList creatureIds;
63     QStringList dominantSpecies;
64 };
```

```

65
66 class EcosystemSystem : public QObject
67 {
68     Q_OBJECT
69 public:
70     explicit EcosystemSystem(QObject* parent = nullptr);
71     ~EcosystemSystem() override;
72
73     bool initialize();
74     void update(float deltaTime);
75
76     // 图鉴系统
77     bool discoverCreature(const QString& creatureId);
78     bool hasDiscovered(const QString& creatureId) const;
79     Creature getCreature(const QString& creatureId) const;
80     QList<Creature> getAllCreatures() const;
81     int getDiscoveredCount() const;
82
83     // 友好度系统
84     FriendshipLevel getFriendshipLevel(const QString& creatureId) const;
85     int getFriendshipPoints(const QString& creatureId) const;
86     void increaseFriendship(const QString& creatureId, int amount, const
108 QString& action);
87     bool canInteract(const QString& creatureId) const;
88
89     // 互动方式
90     bool feedCreature(const QString& creatureId, const QString& itemId);
91     bool petCreature(const QString& creatureId);
92     bool playWithCreature(const QString& creatureId);
93
94     // 伙伴系统
95     bool setCompanion(const QString& creatureId, int slot = 0);
96     QString getCompanion(int slot = 0) const;
97     bool hasCompanion(int slot = 0) const;
98     int getMaxCompanionSlots() const;
99
100    // 伙伴能力
101    QStringList getCompanionAbilities(const QString& creatureId) const;
102    bool useCompanionAbility(const QString& abilityId);
103    float getCompanionBonus(const QString& type) const;
104
105    // 进化系统
106    bool canEvolve(const QString& creatureId) const;
107    bool evolveCreature(const QString& creatureId, const QString&
108 evolutionPath);
109
110    // 生态平衡

```

```

110     float getZoneBalance(const QString& zoneId) const;
111     void affectBalance(const QString& zoneId, float amount);
112     QList<EcosystemZone> getAllZones() const;
113
114     signals:
115         void creatureDiscovered(const QString& creatureId);
116         void friendshipChanged(const QString& creatureId, int level);
117         void companionChanged(int slot, const QString& creatureId);
118         void creatureEvolved(const QString& creatureId, int newStage);
119         void zoneBalanceChanged(const QString& zoneId, float balance);
120
121     private:
122         // 更新友好度等级
123         void updateFriendshipLevel(const QString& creatureId);
124
125         // 更新生态平衡
126         void updateEcosystem(float deltaTime);
127
128         // 成员变量
129         QMap<QString, Creature> m_creatures;
130         QMap<QString, EcosystemZone> m_zones;
131
132         // 伙伴槽位
133         QStringList m_companionSlots;
134         int m_maxCompanionSlots;
135
136         bool m_isInitialized;
137     };

```

6.4 伙伴能力示例

伙伴能力示例

```

1  // 植物型伙伴：治疗光环
2  // 动物型伙伴：攻击辅助
3  // 矿物型伙伴：防御护盾
4  // 能量型伙伴：共鸣增幅
5  // 机械型伙伴：物品收集
6
7  float EcosystemSystem::getCompanionBonus(const QString& type) const
8  {
9      float bonus = 0.0f;
10
11     for (const QString& companionId : m_companionSlots) {
12         if (companionId.isEmpty()) {
13             continue;

```



```
14         }
15
16         const Creature& creature = m_creatures.value(companionId);
17
18         // 根据伙伴类型提供不同加成
19         if (type == "healing" && creature.type == CreatureType::Plant) {
20             bonus += 0.1f * (static_cast<int>(creature.friendshipLevel) + 1);
21         }
22         else if (type == "attack" && creature.type == CreatureType::Animal) {
23             bonus += 0.05f * (static_cast<int>(creature.friendshipLevel) + 1);
24         }
25         else if (type == "defense" && creature.type == CreatureType::Mineral) {
26             bonus += 0.08f * (static_cast<int>(creature.friendshipLevel) + 1);
27         }
28         // ... 更多类型
29     }
30
31     return bonus;
32 }
```

验收标准

- ☐ 编译无错误
- ☐ 生物图鉴系统正常
- ☐ 友好度系统正常
- ☐ 伙伴系统正常
- ☐ 伙伴能力生效
- ☐ 进化系统正常
- ☐ 生态平衡系统正常

八、步骤7：实现技能系统

✨ **目标：**实现完整的技能系统，包括技能树、冷却、效果和升级

修改文件：skillSystem.h / skillSystem.cpp

7.1 技能系统功能

- **技能树：**多条技能路线

- **技能学习**：消耗技能点学习技能
- **技能升级**：提升技能效果
- **冷却系统**：技能冷却管理
- **技能效果**：伤害、治疗、buff等

7.2 技能分类

分类	特点	示例
主动技能	需要手动释放，有冷却	时间减速、星核爆发
被动技能	永久生效，无需释放	生命提升、暴击率提升
终极技能	强力技能，长冷却	时间停止、共鸣爆发
伙伴技能	伙伴提供的技能	治疗光环、护盾

7.3 技能系统实现要点

技能系统实现

```
1  // 技能类型
2  enum class SkillType {
3      Active,      // 主动技能
4      Passive,    // 被动技能
5      Ultimate,   // 终极技能
6      Companion   // 伙伴技能
7  };
8
9  // 技能效果类型
10 enum class SkillEffectType {
11     Damage,      // 伤害
12     Heal,        // 治疗
13     Buff,        // 增益
14     Debuff,      // 减益
15     CrowdControl, // 控制
16     Movement,    // 移动相关
17     Utility      // 功能型
18 };
19
20 // 技能效果
21 struct SkillEffect {
22     SkillEffectType type;
23     float value;
```

```
24     float duration;
25     QString targetType; // self, enemy, area
26     float radius;      // 范围技能的半径
27 };
28
29 // 技能
30 struct Skill {
31     QString id;
32     QString name;
33     QString description;
34     SkillType type;
35
36     int level;
37     int maxLevel;
38     int requiredLevel; // 需要的角色等级
39
40     int energyCost;
41     float cooldown;
42     float currentCooldown;
43
44     bool isUnlocked;
45     bool isEquipped; // 是否已装备（主动技能）
46
47     QList<SkillEffect> effects;
48
49     // 技能树
50     QString parentSkillId;
51     QStringList childSkillIds;
52     int skillPointCost;
53 };
54
55 class SkillSystem : public QObject
56 {
57     Q_OBJECT
58 public:
59     explicit SkillSystem(QObject* parent = nullptr);
60     ~SkillSystem() override;
61
62     bool initialize();
63     void update(float deltaTime);
64
65     // 技能管理
66     bool learnSkill(const QString& skillId);
67     bool upgradeSkill(const QString& skillId);
68     bool hasSkill(const QString& skillId) const;
69     Skill getSkill(const QString& skillId) const;
70     QList<Skill> getAllSkills() const;
```

```

71     QList<Skill> getUnlockedSkills() const;
72
73     // 技能使用
74     bool canUseSkill(const QString& skillId) const;
75     bool useSkill(const QString& skillId);
76     float getSkillCooldown(const QString& skillId) const;
77     float getSkillCooldownPercent(const QString& skillId) const;
78
79     // 技能装备（主动技能槽位）
80     bool equipSkill(const QString& skillId, int slot);
81     bool unequipSkill(int slot);
82     QString getEquippedSkill(int slot) const;
83     int getMaxSkillSlots() const;
84
85     // 技能点
86     int getSkillPoints() const;
87     void addSkillPoints(int points);
88
89     // 被动技能效果
90     float getPassiveBonus(const QString& type) const;
91
92     signals:
93         void skillLearned(const QString& skillId);
94         void skillUpgraded(const QString& skillId, int newLevel);
95         void skillUsed(const QString& skillId);
96         void skillCooldownStarted(const QString& skillId, float cooldown);
97         void skillReady(const QString& skillId);
98         void skillPointsChanged(int points);
99
100    private:
101        // 更新冷却
102        void updateCooldowns(float deltaTime);
103
104        // 执行技能效果
105        void executeSkillEffects(const Skill& skill);
106
107        // 成员变量
108        QMap<QString, Skill> m_skills;
109        QStringList m_equippedSkills; // 装备的主动技能
110        int m_maxSkillSlots;
111        int m_skillPoints;
112
113        bool m_isInitialized;
114    };

```

验收标准

- ☐ 编译无错误
- ☐ 技能学习和升级正常
- ☐ 技能冷却系统正常
- ☐ 技能效果正确执行
- ☐ 技能装备系统正常
- ☐ 被动技能效果生效

九、步骤8：实现AI系统



目标：实现敌人AI系统，包括感知、决策、行为和路径规划

修改文件： aiSystem.h / aiSystem.cpp

8.1 AI系统功能

- 感知系统：视觉、听觉、触觉感知
- 决策系统：状态机或行为树
- 行为系统：巡逻、追击、攻击、逃跑
- 路径规划：简单的寻路算法

8.2 AI行为模式

行为	触发条件	行为描述
待机	无目标、安全	原地等待、播放待机动画
巡逻	无目标、有巡逻点	在巡逻点之间移动
警戒	感知到可疑目标	停下观察、朝向目标方向
追击	发现敌人、在仇恨范围内	追向目标
攻击	目标在攻击范围内	执行攻击动作
逃跑	血量低、敌人太强	逃离目标
受击	受到伤害	受击硬直、击退

8.3 AI系统实现要点

AI系统实现

```
1  // 感知类型
2  enum class SenseType {
3      Sight,        // 视觉
4      Hearing,       // 听觉
5      Touch,        // 触觉（受到攻击）
6      Count
7  };
8
9  // 感知信息
10 struct SenseInfo {
11     SenseType type;
12     QPointF position;
13     float intensity;
14     float timestamp;
15     QString targetId;
16 };
17
18 // AI状态（状态机）
19 enum class AIState {
20     Idle,          // 待机
21     Patrol,        // 巡逻
22     Alert,         // 警戒
23     Chase,         // 追击
24     Attack,        // 攻击
25     Flee,          // 逃跑
26     Hurt,          // 受击
27     Dead,          // 死亡
28     Count
29 };
30
31 class AISystem : public QObject
32 {
33     Q_OBJECT
34 public:
35     explicit AISystem(QObject* parent = nullptr);
36     ~AISystem() override;
37
38     bool initialize();
39     void update(float deltaTime);
40
41     // 注册/注销AI实体
42     void registerAI(const QString& entityId, AIState initialState);
43     void unregisterAI(const QString& entityId);
```

```
44
45     // 感知
46     void addSenseInput(const QString& entityId, const SenseInfo& sense);
47     bool hasTarget(const QString& entityId) const;
48     QString getTarget(const QString& entityId) const;
49
50     // 状态
51     AIState getAIState(const QString& entityId) const;
52     void forceState(const QString& entityId, AIState state);
53
54     // 路径规划
55     void setDestination(const QString& entityId, const QPointF& destination);
56     QPointF getNextWaypoint(const QString& entityId) const;
57
58 private:
59     // 更新单个AI
60     void updateAI(const QString& entityId, float deltaTime);
61
62     // 状态更新
63     void updateIdle(const QString& entityId, float deltaTime);
64     void updatePatrol(const QString& entityId, float deltaTime);
65     void updateAlert(const QString& entityId, float deltaTime);
66     void updateChase(const QString& entityId, float deltaTime);
67     void updateAttack(const QString& entityId, float deltaTime);
68     void updateFlee(const QString& entityId, float deltaTime);
69
70     // 感知处理
71     void processSenses(const QString& entityId);
72
73     // 状态转换
74     void transitionToState(const QString& entityId, AIState newState);
75
76     // 简单寻路
77     void calculatePath(const QString& entityId, const QPointF& target);
78
79     // AI实体数据
80     struct AIEntity {
81         QString id;
82         AIState state;
83         AIState previousState;
84
85         // 感知
86         QList<SenseInfo> recentSenses;
87         QString currentTarget;
88         float targetLastSeenTime;
89
90         // 巡逻
```

```
91         QList<QPointF> patrolPoints;
92         int currentPatrolIndex;
93
94         // 路径
95         QList<QPointF> path;
96         int currentPathIndex;
97
98         // 攻击
99         float attackCooldown;
100        float attackRange;
101        float detectionRange;
102
103        // 状态计时
104        float stateTimer;
105    };
106
107    QMap<QString, AIEntity> m_aiEntities;
108    bool m_isInitialized;
109};
```

验收标准

- ☐ 编译无错误
- ☐ AI状态机正常工作
- ☐ 感知系统正常
- ☐ 巡逻行为正常
- ☐ 追击行为正常
- ☐ 攻击行为正常
- ☐ 路径规划正常

十、步骤9：实现背包和道具系统



目标：实现背包和道具系统，包括物品管理、装备系统、商店和合成

修改文件：inventorySystem.h / inventorySystem.cpp

9.1 背包系统功能

- **物品管理：**拾取、使用、丢弃、堆叠

- **装备系统**：武器、防具、饰品
- **物品分类**：消耗品、装备、材料、任务物品
- **商店系统**：购买、出售
- **合成系统**：材料合成高级物品

9.2 物品类型

类型	子类	说明
消耗品	回复类	药水、食物
	增益类	临时buff道具
	功能类	传送石、钥匙
装备	武器	主手武器
	防具	头盔、护甲、裤子、鞋子
	饰品	戒指、项链
	共鸣器	星核共鸣装备
材料	-	合成材料、升级材料
任务物品	-	任务相关，不可丢弃

9.3 背包系统实现要点

背包系统实现

```
1 // 物品类型
2 enum class ItemType {
3     Consumable,    // 消耗品
4     Equipment,     // 装备
5     Material,      // 材料
6     Quest,         // 任务物品
7     Count
8 };
9
10 // 装备槽位
11 enum class EquipSlot {
12     Weapon,        // 武器
13     Helmet,        // 头盔
14     Armor,         // 护甲
```

```
15     Pants,        // 裤子
16     Boots,        // 鞋子
17     Ring,         // 戒指
18     Necklace,     // 项链
19     Resonator,    // 共鸣器
20     Count
21 };
22
23 // 物品属性加成
24 struct ItemStats {
25     int healthBonus;
26     int energyBonus;
27     int attackBonus;
28     int defenseBonus;
29     float speedBonus;
30     float criticalRateBonus;
31     float criticalDamageBonus;
32     // ... 更多属性
33 };
34
35 // 物品
36 struct Item {
37     QString id;
38     QString name;
39     QString description;
40     ItemType type;
41     QString icon;
42
43     int maxStack;        // 最大堆叠数
44     int price;           // 价格
45     bool isSellable;     // 是否可出售
46     bool isDroppable;    // 是否可丢弃
47
48     // 装备属性（仅装备类型有）
49     EquipSlot equipSlot;
50     ItemStats stats;
51     int requiredLevel;
52
53     // 消耗品效果
54     QString effectType;
55     float effectValue;
56     float effectDuration;
57 };
58
59 // 背包物品（堆叠）
60 struct InventorySlot {
61     QString itemId;
```

```

62     int count;
63     bool isEquipped;
64     int equipSlot; // 如果已装备, 装备在哪个槽位
65 };
66
67 class InventorySystem : public QObject
68 {
69     Q_OBJECT
70 public:
71     explicit InventorySystem(QObject* parent = nullptr);
72     ~InventorySystem() override;
73
74     bool initialize();
75
76     // 背包管理
77     bool addItem(const QString& itemId, int count = 1);
78     bool removeItem(const QString& itemId, int count = 1);
79     int getItemCount(const QString& itemId) const;
80     bool hasItem(const QString& itemId, int count = 1) const;
81     QList<InventorySlot> getInventory() const;
82
83     // 背包容量
84     int getMaxSlots() const;
85     int getUsedSlots() const;
86     bool isFull() const;
87
88     // 使用物品
89     bool useItem(const QString& itemId);
90     bool canUseItem(const QString& itemId) const;
91
92     // 装备系统
93     bool equipItem(const QString& itemId);
94     bool unequipItem(EquipSlot slot);
95     QString getEquippedItem(EquipSlot slot) const;
96     bool isEquipped(const QString& itemId) const;
97
98     // 属性加成
99     ItemStats getTotalEquipmentStats() const;
100
101     // 金币
102     int getGold() const;
103     void addGold(int amount);
104     bool spendGold(int amount);
105
106     // 商店
107     bool buyItem(const QString& itemId, int count = 1);
108     bool sellItem(const QString& itemId, int count = 1);

```

```
109
110     // 合成
111     bool canCraft(const QString& recipeId) const;
112     bool craftItem(const QString& recipeId);
113
114     signals:
115         void itemAdded(const QString& itemId, int count);
116         void itemRemoved(const QString& itemId, int count);
117         void itemUsed(const QString& itemId);
118         void itemEquipped(const QString& itemId, int slot);
119         void itemUnequipped(int slot);
120         void goldChanged(int newGold);
121
122     private:
123         // 查找空槽位
124         int findEmptySlot() const;
125         int findItemSlot(const QString& itemId) const;
126
127         // 成员变量
128         QMap<QString, Item> m_itemDatabase; // 物品数据库
129         QList<InventorySlot> m_inventory;    // 背包物品
130         int m_maxSlots;
131
132         // 装备槽位
133         QMap<EquipSlot, QString> m_equipment;
134
135         // 金币
136         int m_gold;
137
138         bool m_isInitialized;
139     };
```

验收标准

- ☐ 编译无错误
 - ☐ 物品添加和移除正常
 - ☐ 物品堆叠正常
 - ☐ 使用物品功能正常
 - ☐ 装备系统正常
 - ☐ 属性加成正确
 - ☐ 商店系统正常
-

十一、步骤10：实现任务系统



目标：实现任务系统，包括任务链、进度追踪、奖励和对话

修改文件：questSystem.h / questSystem.cpp

10.1 任务系统功能

- **任务类型：**主线、支线、日常、隐藏
- **任务目标：**击杀、收集、探索、对话
- **任务链：**多步骤任务
- **奖励系统：**经验、金币、物品
- **对话系统：**NPC对话、选项

10.2 任务类型

类型	特点	示例
主线任务	推动剧情、不可错过	寻找星核、揭开真相
支线任务	可选、丰富世界观	帮助NPC、探索秘密
日常任务	每日刷新、重复做	收集材料、击杀怪物
隐藏任务	需要特定条件触发	秘密、彩蛋

10.3 任务系统实现要点

任务系统实现

```
1 // 任务类型
2 enum class QuestType {
3     Main,          // 主线
4     Side,          // 支线
5     Daily,         // 日常
6     Hidden,        // 隐藏
7     Count
8 };
9
10 // 任务状态
11 enum class QuestState {
12     NotAvailable,  // 未解锁
```

```
13     Available,          // 可接取
14     InProgress,         // 进行中
15     Completed,          // 已完成
16     TurnedIn,           // 已交付
17     Failed              // 失败
18 };
19
20 // 任务目标类型
21 enum class QuestObjectiveType {
22     Kill,                // 击杀
23     Collect,             // 收集
24     Explore,             // 探索
25     Talk,               // 对话
26     Interact,           // 交互
27     Escort,             // 护送
28     Count
29 };
30
31 // 任务目标
32 struct QuestObjective {
33     QString id;
34     QuestObjectiveType type;
35     QString description;
36
37     QString targetId;     // 目标ID (怪物ID、物品ID等)
38     int targetCount;      // 需要数量
39     int currentCount;     // 当前进度
40
41     bool isOptional;      // 是否可选
42     bool isCompleted;
43 };
44
45 // 任务奖励
46 struct QuestReward {
47     int exp;
48     int gold;
49     QList<QString> itemIds;
50     QList<int> itemCounts;
51     QString unlocksQuest; // 解锁的任务
52     QString unlocksArea;  // 解锁的区域
53 };
54
55 // 任务
56 struct Quest {
57     QString id;
58     QString name;
59     QString description;
```

```

60     QuestType type;
61     QuestState state;
62
63     QString giver_npc_id;    // 给予任务的NPC
64     QString turn_in_npc_id;  // 交付任务的NPC
65
66     int required_level;      // 需要等级
67     QString required_quest;  // 前置任务
68
69     QList<QuestObjective> objectives;
70     QuestReward rewards;
71
72     // 对话
73     QString accept_dialogue;
74     QString in_progress_dialogue;
75     QString complete_dialogue;
76 };
77
78 // 对话选项
79 struct DialogueOption {
80     QString id;
81     QString text;
82     QString next_dialogue_id;
83     QString action_type;    // accept_quest, give_item, etc.
84     QString action_data;
85 };
86
87 // 对话
88 struct Dialogue {
89     QString id;
90     QString speaker;
91     QString text;
92     QList<DialogueOption> options;
93 };
94
95 class QuestSystem : public QObject
96 {
97     Q_OBJECT
98 public:
99     explicit QuestSystem(QObject* parent = nullptr);
100     ~QuestSystem() override;
101
102     bool initialize();
103     void update(float delta_time);
104
105     // 任务管理
106     bool accept_quest(const QString& quest_id);

```

```
107     bool abandonQuest(const QString& questId);
108     bool completeQuest(const QString& questId);
109     bool turnInQuest(const QString& questId);
110
111     QuestState getQuestState(const QString& questId) const;
112     Quest getQuest(const QString& questId) const;
113     QList<Quest> getAllQuests() const;
114     QList<Quest> getActiveQuests() const;
115     QList<Quest> getAvailableQuests() const;
116     QList<Quest> getCompletedQuests() const;
117
118     // 任务进度
119     void updateObjective(const QString& questId, const QString& objectiveId,
120 int progress = 1);
121     float getQuestProgress(const QString& questId) const;
122     bool isQuestComplete(const QString& questId) const;
123
124     // 任务追踪
125     void setTrackedQuest(const QString& questId);
126     QString getTrackedQuest() const;
127
128     // 对话系统
129     Dialogue getDialogue(const QString& dialogueId) const;
130     bool selectDialogueOption(const QString& dialogueId, const QString&
optionId);
131
132 signals:
133     void questAccepted(const QString& questId);
134     void questAbandoned(const QString& questId);
135     void questCompleted(const QString& questId);
136     void questTurnedIn(const QString& questId);
137     void objectiveUpdated(const QString& questId, const QString& objectiveId);
138     void questTrackedChanged(const QString& questId);
139
140 private:
141     // 检查任务是否可接取
142     bool canAcceptQuest(const QString& questId) const;
143
144     // 更新任务状态
145     void checkQuestCompletion(const QString& questId);
146
147     // 发放奖励
148     void giveRewards(const QuestReward& rewards);
149
150     // 成员变量
151     QMap<QString, Quest> m_requests;
152     QMap<QString, Dialogue> m_dialogues;
```



```
152
153     QString m_trackedQuest;
154
155     bool m_isInitialized;
156 };
```

验收标准

- ☐ 编译无错误
- ☐ 任务接取和交付正常
- ☐ 任务进度更新正常
- ☐ 任务目标类型都支持
- ☐ 奖励发放正常
- ☐ 任务链正常工作
- ☐ 对话系统正常

十二、步骤11：游戏系统集成测试



目标：创建完整的游戏测试场景，验证所有游戏系统，进行平衡性测试

11.1 创建测试关卡

创建一个专门的游戏测试关卡，包含：

- 完整的瓦片地图（有地形、障碍、平台）
- 多种敌人类型（普通、远程、精英）
- 可收集物品和宝箱
- NPC和任务发布者
- 星核碎片和生态生物
- 存档点和传送点

11.2 功能测试清单

玩家系统

- ☐ 玩家移动流畅、响应及时

- ☐ 跳跃和落地正常
- ☐ 攻击动作和判定正常
- ☐ 受击反馈和无敌帧正常
- ☐ 升级和属性提升正常
- ☐ 死亡和复活正常

战斗系统

- ☐ 伤害计算正确
- ☐ 暴击系统正常
- ☐ 连击系统正常
- ☐ 击退效果正常
- ☐ 战斗状态切换正常

三大核心系统

- ☐ 时间减速/加速效果明显
- ☐ 时间回溯功能正常
- ☐ 悖论系统正常工作
- ☐ 星核共鸣能力正常
- ☐ 共鸣反噬效果正常
- ☐ 生态伙伴跟随正常
- ☐ 伙伴能力生效

其他系统

- ☐ 技能释放和冷却正常
- ☐ 敌人AI行为正常
- ☐ 背包和装备正常
- ☐ 任务接取和完成正常

11.3 平衡性测试

测试项目	目标	测试方法
角色成长曲线	平滑、有成就感	1-10级每级体验

敌人难度	有挑战但不挫败	同级敌人战斗测试
经济系统	金币获取和消耗平衡	1小时游戏测试
技能强度	各技能都有用	全技能实战测试
核心系统强度	强力但有代价	三大系统对比测试

11.4 压力测试

- 大量敌人同屏测试（20+敌人）
- 大量粒子效果测试
- 长时间游戏测试（2小时以上）
- 快速切换场景测试
- 频繁使用核心系统测试

十三、第四阶段总结



第四阶段完成标志：所有游戏系统实现完成，三大核心特色系统功能完整，有可玩的测试关卡，平衡性基本达标

完成清单

- ☐ 玩家角色系统完善
- ☐ 敌人系统实现
- ☐ 战斗系统实现
- ☐ 时间褶皱系统实现
- ☐ 星核共鸣系统实现
- ☐ 生态共生系统实现
- ☐ 技能系统实现
- ☐ AI系统实现
- ☐ 背包道具系统实现
- ☐ 任务系统实现
- ☐ 集成测试通过
- ☐ 平衡性基本达标

三大核心系统亮点



时间褶皱

- 时间减速/加速/暂停
- 时间回溯（最多5秒）
- 悖论反噬机制
- 玩家免疫时间效果



星核共鸣

- 7种共鸣类型
- 环境交互能力
- 共鸣等级成长
- 反噬风险机制



生态共生

- 生物图鉴收集
- 友好度培养
- 伙伴协同作战
- 生态平衡系统

下一步：第五阶段

第五阶段将进入内容制作和UI系统开发，实现：

- 完整的UI界面（主菜单、HUD、背包、技能树等）
- 游戏音效和背景音乐
- 多个关卡和区域设计
- 剧情和对话内容
- 更多敌人和Boss
- 更多道具和装备



恭喜！ 第四阶段游戏系统开发完成后，你将拥有一个玩法完整、特色鲜明的银河恶魔城游戏原型！三大核心系统将成为《星溯旅人》最独特的玩法体验！

（注：文档部分内容可能由 AI 生成）